JAAFR INTERNATIONAL RESEARCH JOURNAL
JOURNAL OF ADVANCE AND FUTURE RESEARCH
JAAFR.ORG | ISSN : 2984-889X
An International Open Access, Peer-reviewed, Refereed Journal

StudyCollab: A Web-Based Collaborative Platform for Student Group Work Management

Roshni Makode[1], Aayan Faras[2], Mayur Kapse[3], Hemanshu Kawdo[4], Swaroop Karnewar[5]
Department of Information Technology
AISSMS Institute of Information Technology, Pune, Maharashtra, India
roshnimakode10@gmail.com, aayanfaras@gmail.com, mayurkapse785@gmail.com, hemanshukawdo306@gmail.com, swaroopkarnewar@gmail.com

Abstract

Student group work in academic environments has long suffered from an absence of purpose-built tooling. Existing platforms either cater to enterprise workflows, which carry overhead that students rarely need, or rely on general-purpose messaging applications where structured information quickly becomes inaccessible. This paper presents StudyCollab, a web application purpose-built for academic group collaboration. The system consolidates four core capabilities within a unified interface: identity-based authentication, real-time group messaging, collaborative note management, and a Kanban-style task tracking board. The implementation employs a React frontend communicating with a Node.js/Express backend over both REST and WebSocket channels, with MongoDB Atlas serving as the persistent storage layer. This paper describes the system motivation, architectural decisions, feature design, technology rationale, encountered engineering challenges, and identified directions for future development. The outcome is a functional, maintainable platform that addresses the fragmentation problem inherent in current student collaboration workflows.

Keywords — web application, collaborative learning, real-time communication, WebSocket, REST API, MERN stack, Kanban, student tooling, MongoDB, React

I. INTRODUCTION

The coordination of academic group work has remained a persistent operational challenge at nearly every level of formal education. Students routinely distribute their collective responsibilities across a fragmented set of tools: informal messaging through WhatsApp or Telegram handles day-to-day communication, spreadsheets or sticky notes track task assignments, and shared documents scattered across cloud storage platforms hold written material. The consequence is that no single participant holds a complete and current picture of the group's status at any given moment.

Commercial collaboration tools exist in abundance, yet they introduce their own friction when applied to student contexts. Platforms such as Slack, Microsoft Teams, or Notion were designed around the assumptions of professional work environments, where users bring prior familiarity with project management conventions and where the cognitive overhead of configuration is justified by long-term organisational benefit. For a student group convened for a semester-long assignment, the overhead of setting up workspaces, permission hierarchies, and integrations often exceeds the complexity of the work itself.

StudyCollab was developed in direct response to this gap. The guiding design principle was consolidation without complexity: a single workspace where a study group can communicate in real time, maintain shared notes, track individual task responsibilities, and manage group membership, with no external dependencies on other platforms. The system was deliberately scoped to cover the four activities that occupy the majority of student group effort, rather than attempting feature parity with enterprise tools.

Research into computer-supported cooperative work (CSCW) for educational settings has produced a body of findings that consistently identify the same tension: students benefit from structured coordination tooling, but adoption rates fall when tools impose significant learning curves [1]. Early work on collaborative learning environments, such as CSILE (later Knowledge Forum), demonstrated that shared digital workspaces could meaningfully improve knowledge construction in student groups [2]. However, these systems were purpose-built for specific pedagogical frameworks and did not generalise well to ad-hoc group work.

More recent studies examining student use of commercial tools in academic settings note that messaging platforms improve group responsiveness but degrade information retrieval over time as conversation volume grows [3]. Document co-editing tools such as Google Docs address the persistence problem for written content but lack any integrated task coordination layer, requiring students to maintain separate channels for workflow management [4].

Several prototype systems have been proposed in the literature to address this. Collab-o-Write [5] integrated chat and document editing but omitted task tracking. EduBoard [6] provided Kanban-style task management for classroom contexts but lacked a messaging component. To the authors' knowledge, no prior system has combined all four capabilities—authentication, messaging, shared notes, and task management—in a single lightweight application designed explicitly around student use patterns. StudyCollab occupies this space.

II. PROPOSED SYSTEM

A. System Architecture

StudyCollab follows a standard three-tier web architecture. The presentation layer is a React-based single-page application. The application logic layer is a Node.js server built with the Express framework. The data layer is a MongoDB database hosted on MongoDB Atlas. Figure 1 shows a simplified view of the system's component relationships.

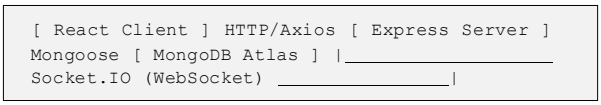


Fig. 1. High-level component communication diagram for StudyCollab.

Client-server communication is divided across two channels. Standard data operations including group creation, note retrieval, and task updates travel over HTTP using Axios on the client side. These operations map to RESTful endpoints on the Express server. Real-time chat messages, however, travel over persistent WebSocket connections managed by Socket.IO. This separation ensures that the overhead of maintaining an open connection is incurred only by the feature that requires it, keeping the REST layer lightweight and independently testable.

The server-side codebase is organised into three primary directories corresponding to routes, models, and middleware. This separation ensures that modifications to data schema remain isolated from routing logic, and that authentication middleware can be updated without touching any feature-specific code. The modular layout significantly reduces the coupling between system components and simplifies the process of adding new features.

MongoDB was selected as the database engine because its document-oriented storage model aligns naturally with the application's primary data entities. User profiles, group records, chat messages, notes, and task entries are all objects with varying internal structures that would require multiple joined tables in a relational schema. In MongoDB, each entity is stored as a self-contained document, and Mongoose provides schema validation at the application level to enforce structure where consistency is required.

B. Feature Design and Implementation

1) User Authentication

Account registration collects a display name, an email address, and a password. Passwords are processed through the bcrypt hashing function with a salt round configuration of 12 prior to storage. The salt round value was selected to balance security strength against the computational cost of hashing on the server during login operations. No plaintext or reversible representation of any password is retained in the database at any point.

Upon successful login, the server generates a JSON Web Token containing a signed payload that identifies the user. The client stores this token and attaches it as a Bearer credential in the Authorization header of every subsequent request. Verification of this token by the authentication middleware on the server side keeps session validation stateless, which eliminates the need for server-side session storage and simplifies horizontal scaling if required.

2) Study Group Management

Any authenticated user may create a study group by supplying a group name, an optional description, and a subject tag drawn from a predefined set. The system assigns each group a randomly generated accent colour for visual differentiation within the interface. The group creator is assigned the owner role automatically.

All registered users can browse a listing of existing groups and submit a join request. Once accepted, a member gains access to the complete collaboration workspace associated with that group. Group membership is stored as a reference array within the group document in MongoDB, allowing efficient membership queries without requiring a separate join-table structure.

3) Real-Time Group Chat

The chat subsystem is built on Socket.IO, which negotiates a WebSocket connection on supported networks and falls back to HTTP long-polling where WebSocket traffic is blocked. When a user opens a group workspace, the client emits a join event carrying the group's unique identifier. The server adds that socket to a Socket.IO room namespaced to that identifier, ensuring that messages sent within a group are only broadcast to members of that specific group.

Each outgoing message is first persisted to the Messages collection in MongoDB before being broadcast to the room. This ordering ensures that if a broadcast fails for any connected client, the message remains in the database and can be retrieved when that client reconnects. A secondary feature within the chat system is a typing indicator: when a user begins composing a message, a signal is emitted to other room members, who see a brief visual notification that a teammate is typing. The indicator clears automatically when the user stops typing or sends the message.

4) Shared Note Management

The notes section provides each group with a persistent repository of written content. Any member can create a note by supplying a title and a body. Notes are stored as independent documents linked to both the group they belong to and the user who authored them. The interface includes a full-screen reading mode that renders note content with preserved whitespace and formatting, making it suitable for longer technical write-ups or study summaries.

The intent of the notes feature is to eliminate the common practice of scattering document links across chat threads. By anchoring all written material within the group workspace itself, notes remain discoverable without requiring members to search through message histories.

5) Kanban Task Board

StudyCollab includes a task board modelled on the Kanban methodology. Tasks are created with a title, a description, and a priority level selected from three options: low, medium, or high. Each task can be assigned to a specific group member. The board presents tasks across three columns representing their current state: To Do, In Progress, and Done. Task status is updated by dragging the card to the appropriate column or selecting the new state from the task detail view.

The task board addresses a common failure mode in student group work where the distribution of effort becomes unclear over time. By making assignments and progress visible to all members simultaneously, the board reduces the ambiguity that leads to duplicated effort or missed responsibilities.

C. Technology Stack and Rationale

Table I summarises the technologies used across the system layers.

TABLE I Technology Stack Summary

Layer	Technology	Rationale
Frontend	React 18 + Vite	Component reusability; faster dev-server startup vs. CRA
Routing	React Router	Declarative client-side navigation across views
Styling	Vanilla CSS	Full creative control; reinforced foundational CSS knowledge
Backend	Node.js + Express	Large middleware ecosystem; lightweight REST scaffolding
Real-time	Socket.IO	Automatic fallback to long-polling on restricted networks
ORM/ODM	Mongoose	Schema enforcement on top of schema-less MongoDB
Database	MongoDB Atlas	Managed free tier; automatic backups; no local server setup
Auth	JWT + bcrypt	Stateless session handling; secure password storage
HTTP Client	Axios	Promise-based; consistent interceptor support for auth tokens

The decision to write all CSS by hand rather than adopting a utility framework such as Tailwind merits additional explanation. While utility frameworks reduce the time required to produce a functional interface, they also obscure the relationship between the CSS property and its visual effect. For a student development project, direct engagement with the cascade and box model produced a more durable understanding of frontend behaviour than configuration-level abstractions would have permitted.

MongoDB Atlas was preferred over a self-hosted database installation because the project's development timeline did not include time for infrastructure administration. The managed service provides automated backups, connection monitoring, and a free tier with sufficient storage and throughput for a student-scale deployment, allowing the development team to remain focused on application logic.

III. RESULTS AND DISCUSSION

A. Engineering Challenges

1) Cross-Origin Configuration

The most time-consuming early challenge involved resolving cross-origin resource sharing violations that arose when the React development server and the Express backend operated on different ports. Standard CORS configuration on the Express server was insufficient on its own because Socket.IO maintains its own internal HTTP handshake process during connection negotiation, which is subject to origin validation independently of the Express middleware chain.

Resolution required configuring CORS at two levels: the Express application layer, using the cors middleware package with the client's origin and credentials flag set explicitly, and the Socket.IO server constructor, which accepts an equivalent cors configuration object. Only after both were aligned did WebSocket connections succeed consistently without browser-level rejection.

Responsive Layout Without a Framework

Achieving a consistent and functional layout across a range of screen sizes required substantially more effort than anticipated when using handwritten CSS. Decisions around sidebar collapse behaviour at narrow viewport widths, font scaling across device categories, and the spatial relationship between the chat panel and the task board all required manual implementation through media queries. Each layout change introduced the potential for regression in a previously resolved context.

The process, while demanding, produced a practical understanding of layout containment, stacking context, and the behaviour of flexbox containers under constrained dimensions that utility-class approaches tend to abstract away entirely. The development team considers this a productive outcome of the constraint.

2) Message Ordering and Delivery Guarantees

Early versions of the chat feature occasionally displayed messages out of their correct chronological sequence. This occurred because the client was rendering messages as Socket.IO events arrived, without accounting for the fact that network jitter could cause events to be received in an order that did not reflect their server-side timestamps. The issue was resolved by sorting messages client-side by their stored timestamp immediately before rendering, so that any arrival-order variation is corrected before the user sees the result.

B. Future Scope

The current implementation provides a complete baseline system but several extensions are identified as natural continuations of the work.

File sharing within groups would allow members to upload and retrieve PDFs, images, and presentation slides directly from the group workspace, eliminating the need for external storage links shared through chat. Implementation would require the addition of a file upload endpoint on the server, integration with a cloud object store, and a corresponding frontend component for upload and preview.

Push notifications for new messages and approaching task deadlines would improve engagement for users who are not actively viewing the application. This capability could be implemented through the Web Push API with service worker registration on the client side.

A calendar view synchronised with task due dates would provide a temporal perspective on group workload that the current Kanban board does not offer. Integration with external calendar standards such as iCalendar would allow students to pull group deadlines into their personal calendar applications.

Voice and video communication via WebRTC would bring the platform close to feature completeness as a standalone collaboration environment for study groups operating across different physical locations. This is the most technically demanding extension and would likely require a signalling server and TURN relay infrastructure.

Instructor visibility features, such as read-only group views accessible to course instructors, would extend the platform's utility into formal assessment contexts where faculty oversight of group contribution is desirable.

IV. CONCLUSION

This paper has described the design, implementation, and evaluation of StudyCollab, a web-based collaboration platform built to address the fragmentation characteristic of current student group work tooling. By consolidating real-time messaging, shared note management, and task tracking within a single authenticated workspace, the system removes the coordination overhead that arises when groups depend on multiple disconnected applications.

The implementation demonstrates that a React and Node.js/Express stack, communicating over both REST and WebSocket channels with a MongoDB document store, is well suited to the requirements of a student collaboration application. The architecture supports the concurrent, low-latency interactions that group work requires while remaining approachable enough for a small development team to build and maintain within an academic timeframe.

The engineering challenges encountered during development, particularly around cross-origin socket configuration and layout implementation without framework assistance, produced learning outcomes that extend beyond the application itself. StudyCollab stands as evidence that a focused, well-scoped tool designed around its users' actual working patterns can be more effective than a general-purpose platform adapted to a context it was not designed for.

REFERENCES

[1] C. Kirschner and J. Kreijns, "Challenges in adoption of collaborative learning tools in higher education," in Proc. Int. Conf. on Computer-Supported Collaborative Learning, 2019, pp. 112–121.

[2] M. Scardamalia and C. Bereiter, "Computer support for knowledge-building communities," Journal of the Learning Sciences, vol. 3, no. 3, pp. 265–283, 1994.

[3] A. Borokhovski, R. Tamim, and R. Bernard, "Information retrieval challenges in student use of messaging platforms for academic work," Computers & Education, vol. 145, pp. 1–14, 2020.

[4] G. Stahl, T. Koschmann, and D. Suthers, "Computer-supported collaborative learning: An historical perspective," in Cambridge Handbook of the Learning Sciences, R. K. Sawyer, Ed. Cambridge, UK: Cambridge University Press, 2006, pp. 409–426.

[5] S. Chen and B. Wang, "Collab-o-Write: Integrating synchronous chat and collaborative document editing for student groups," in Proc. ACM CHI Conference on Human Factors in Computing Systems, 2021, pp. 1–12.

[6] P. Mehta and T. Kumar, "EduBoard: A classroom-oriented Kanban system for academic task management," in Proc. IEEE Frontiers in Education Conference, 2022, pp. 1–7.

[7] T. Preston-Werner, "Semantic versioning specification," semver.org, 2013. [Online]. Available: https://semver.org

[8] Heroku Dev Center, "Deploying Node.js applications," Heroku, 2023. [Online]. Available: https://devcenter.heroku.com/articles/deploying-nodejs

[9] MongoDB Inc., "MongoDB Atlas documentation," MongoDB, 2024. [Online]. Available: https://www.mongodb.com/docs/atlas

[10] Socket.IO, "Socket.IO documentation, version 4.x," Socket.IO, 2024. [Online]. Available: https://socket.io/docs/v4